PROJECT REPORT

CODELOCK – Algorithmic Escape Room v1.0

Students Name	THASBEEHA.N , SWETHA.B , ROHINI
Roll Number	[R
Department	Electronics and Communication Engineering
Academic Year	2026 – 2027
Programming Language	C++ (C++17)
Project Version	v1.0
GitHub Repository	github.com/your-username/codelock

A C++ Data Structures & Algorithms Puzzle Game

1. Introduction

CODELOCK is a puzzle game written in C++ in which the player is locked inside a virtual escape room and must solve a series of 5 puzzles before a countdown timer reaches zero. Every room is built around a different data structure or algorithm — Stack, Queue, Binary Search Tree, Graph traversal and Dijkstra's shortest path algorithm — making puzzle solving unlocking the lock sequence.

This project was developed as part of the Data Structures curriculum. Its aim is to show how data structures and algorithms are practical tools for solving problems, rather than separate topics in programming. The game runs in the terminal and uses only C++ standard library (no HTML, JavaScript or Python is involved).

2. Objectives

- Build a complete, playable escape room game using only C++.
- Implement a Stack, Queue, Binary Search Tree, Min-Heap and Graph from scratch and use them in the puzzles.
- Apply Breadth-First Search, Depth-First Search and Dijkstra's algorithm to realistic traversal and shortest-path problems.
- Provide five progressively harder levels, each unlocked by solving the previous one.
- Add a countdown timer, lives, hints and error validation.
- Track score and attempts, remember the best score between runs, and show story play-through.
- Keep the design modular and object-oriented, and verify it with unit tests and scripted play-throughs.

3. Project Description

CODELOCK is a standalone C++17 console application. The player types answers at a prompt. Every answer is automatically checked by running the relevant data structure or algorithm, so the correct result is always computed by real code rather than stored as a fixed string. Where a puzzle has several valid answers (for example, more than one traversal order), any valid answer is accepted.

3.1 Game Flow

- **Start Game:** The player enters a name and presses ENTER. The timer, score, lives and level counter are initialized.
- **Display Puzzle:** The room is described and a clue or logical challenge is shown. The relevant C++ data structure algorithm processes the input to work out the correct answer.
- **Check Answer:** A correct answer unlocks the next level; an incorrect answer costs a life and the player re-tries.
- **Escaped or Game Over:** The final score, time taken and levels completed are displayed.

3.2 The five levels

Level	Name / Challenge	C++ concept	How the answer is checked
1	The Key Vault – retrieve keys in order	Stack (LIFO)	Push/Pop steps are processed on a Stack; the player must type the keys in the order they are popped.
2	The Waiting Gate – process requests in order	Queue (FIFO)	Enqueue/Dequeue steps move patients into a Queue; the player types who is served first to last.
3	The Hidden Code – find a secret value in a tree	Binary Search Tree	Numbers are inserted into a BST; the player types key/node visited while searching for 35.
4	The Maze – navigate connected rooms	Graph — BFS / DFS	Part 1: DFS counts reachable rooms. Part 2: BFS finds the route with fewest door doors.
5	The Final Escape – cheapest path to the exit	Graph — Dijkstra + MinHeap	Part 1: Minimum total cost. Part 2: exact shortest path node sequence.

3.3 Cost of the structures and algorithms used

Structure / Algorithm	Operation	Time complexity
Stack (array-based)	push, pop, top	O(1)
Queue (circular array)	enqueue, dequeue, front	O(1)
Binary Search Tree	insert, search	O(h); O(log n) on average, O(n) worst case
Breadth-First Search	count reachable nodes	O(V + E)
Depth-First Search	count reachable nodes	O(V + E)
Dijkstra's algorithm	shortest path (Min-Heap)	O((V + E) log V)

4. System Features

4.1 Puzzle Rooms

- Five levels of increasing difficulty; a level can only be played after the previous one is solved.
- Levels 4 and 5 are two-part puzzles (explore, then escape: the first gives the price, then the route).
- When room 3 (BST) is cleared, it shows a in-order traversal of the tree, for example: the in-order walk of the tree in Level 3 and an optional route found by BFS or Dijkstra in Level 4 and 5.

4.2 Timer, Lives and Score

- Countdown timer for the whole game (five minutes by default; configurable with flags). The timer is shown before every question. An answer that arrives after the clock reaches zero is rejected and the game ends.
- Three lives. Every wrong answer loses one life; with no lives left the room locks down.
- Each room is worth up to 100 points. Every wrong answer or hint deducts 10 points from that room, but a solved room is never worth less than 20.
- On escape the player gains 1 point per second remaining and 50 points per life remaining. The best-possible score is 950.

4.3 Hints and Answer Validation

- Typing hint shows a clue for the current puzzle (costs 10 points); typing quit gives up.
- Answers are not case-sensitive, and items may be separated by spaces, commas or arrows (for example A-C-E-B, A C E B, A->C->E->B are all treated identically).
- Route answers are validated door by door against the graph, so any correct shortest or shortest-cost path is accepted, not just one fixed string.

4.4 Results and High Score

- Victory screen with the score breakdown (room points, time bonus, lives bonus), time taken, wrong answers, hints used and lives left.
- Game-over screen that states the reason (time up, no lives left, or player quit) together with the progress made.
- The best score is saved in a small text file and reported at the end of each game.

5. System Modules

Module	Class / File	Responsibility
Entry point	main.cpp	Reads command-line options (time limit etc.) and starts the game.
Game controller	Game.cpp / Game.h	Owns the 5 levels, runs the game loop and computes the final score.
Level base class	Level.cpp / Level.h	Abstract class with the shared prompt-loop, clean check, lives, hints, quit.
Level 1	KeyVaultLevel.cpp	Stack puzzle.
Level 2	WaitingGateLevel.cpp	Queue puzzle.
Level 3	HiddenCodeLevel.cpp	Binary Search Tree puzzle.
Level 4	MazeLevel.cpp	Graph puzzle using DFS and BFS.
Level 5	FinalEscapeLevel.cpp	Weighted graph puzzle using Dijkstra and MinHeap.
Game state	GameState.cpp / .h	Lives, score, level progress, mistake and hint counter.

Module	Class / File	Responsibility
Timer	Timer.cpp / .h	Countdown timer based on a steady (monotonic) clock.
Stack	Stack.h	Template stack based on std::vector.
Queue	Queue.h	Template circular queue (FIFO).
Binary Search Tree	BST.cpp / BST.h	Insert, search, path to node, in-order walk.
Min Heap	MinHeap.h	Template priority queue used by the priority queue shortest path.
Graph	Graph.cpp / Graph.h	Weighted undirected graph with adjacency list matrix.
Algorithms	Algorithms.cpp / .h	BFS reachable count, BFS shortest path, DFS, Dijkstra shortest path.
User interface	ConsoleUI.cpp / .h	Banners, room titles, headers, status line, victory and game over screens.
Score board	ScoreBoard.cpp / .h	Loads and saves the best score on disk.
Test helpers	tests.cpp	Three tests: form, grade validation, automated playthroughs.
Tree helpers	StringUtil.cpp / .h	Trim, lower-case, split, string-of-answers.
Constants	Constants.h	Time limits, lives, point values.
Test script	run.sh	Build, run and run test targets (make, cmake, build).

6. Technologies Used

Technology	Version	Purpose
C++	C++17	Core programming language.
Data Data Structures	Stack, Queue, BST, MinHeap, Graph	Written from scratch to drive the puzzles.
C++ Standard Library	Built-in	std::vector, std::string, std::iostream, chrono, etc.
GNU g++	Built-in	Compiler: built with -Wall -Wextra -Werror to ensure no warnings.
POSIX Make	Built-in	Build, run and test targets (make, make test).
Shell script	sh/bash	Scripted play-through test.
Git/GitHub	Latest	Version control and source hosting.

7. Project Folder Structure

```
codelock/ ├── src/ | ├── main.cpp | ├── Game.cpp / Game.h | ├── GameState.cpp / GameState.h | ├
```

The file bestscores.txt is created the first time a score is saved and is not kept in version control.

8. Sample Output / Screenshots

The figures below show the real output of the program. Figures 1 to 7 come from one complete game played by Thasbeeha (with one wrong answer and one hint); Figure 8 comes from a second run in which the player runs out of lives.

8.1 Start Screen

The game opens with the title, the rules, the time limit, the number of lives and the scoring scheme, then asks for the player's name.



Figure 1 — Start screen with rules of the game

8.2 Level 1 – The Key Vault (Stack)

The robot arm PUSH and POP steps are listed. The player must give the keys in the order they are popped. The status line shows the time left, the lives and the score.



Figure 2 — Level 1 passed on first attempt

8.3 Wrong Answer and Lives

A wrong answer deducts 10 points and one life. The player tries again and the room is worth 90 instead of 100 points.

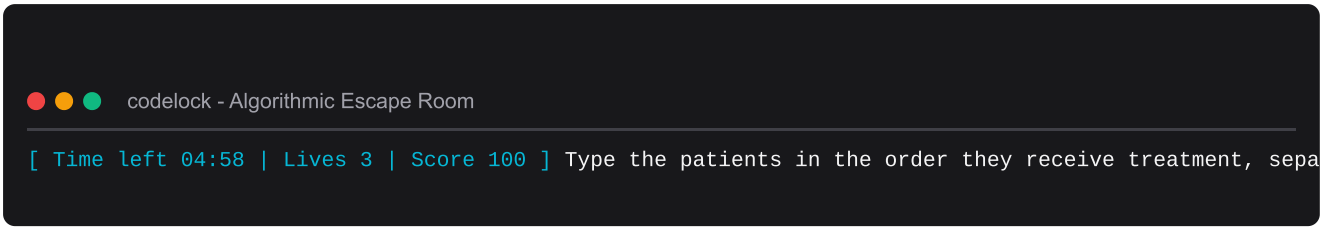


Figure 3 — Wrong answer deducts one life

8.4 Hints

Typing hint in level 3 shows a clue and deducts 10 points. After the room is solved the program prints the in-order traversal of the tree, which is sorted.



Figure 4 — Using a hint in Level 3

8.5 Level 4 – The Maze (Graph, DFS and BFS)

Part 1 asks how many rooms can be reached from room A (found with DFS); rooms I and J are walled off). Part 2 asks for a route from A to H with the fewest doors (found with BFS).



Figure 5 — The maze solved in two parts

8.6 Level 5 – The Final Escape (Dijkstra)

The tunnels have energy costs. The player first gives the minimum total cost and then a route with that cost, computed by Dijkstra's algorithm with a priority queue.



Figure 6 — The final level: weighted graph path

8.7 Victory Screen

When all five rooms are solved the game shows the score breakdown. In this run the player escaped with 283 seconds remaining and 2 lives left.



Figure 7 — Victory screen with full score breakdown

8.8 Game Over Screen

Three wrong answers end the game. The reason, the score and the progress made are shown.

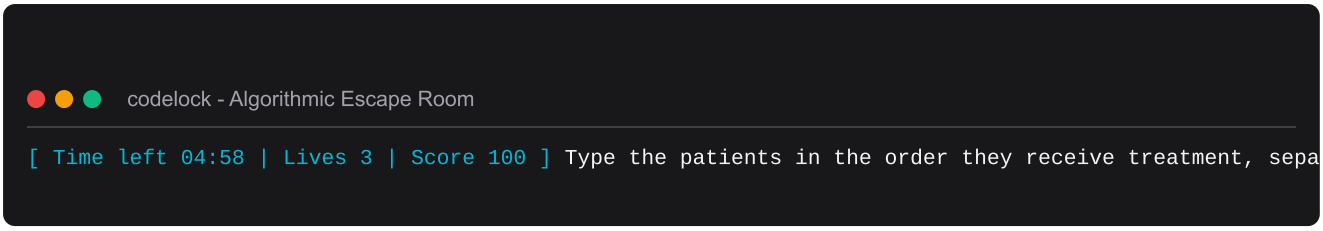


Figure 8 — Game over after the third mistake

8.9 Test Results

The project ships with two kinds of automated tests (make test). All of them passed on the final build.

Test suite	What it checks	Result
Unit tests (test_structures.cpp)	Stack, Queue, BST, Min-Heap, Graph, BFS, DFS, Dijkstra, path validation, scoring, ScoreBoard file saving.	68 checks passed
Play-through test (test_playthrough.sh)	Perfect escape, wrong-answer and hint-penalty paths, time-out game over, out-of-lives game over, high-score file creation, bad command-line input handling, player-quit, score-board limit, high score updated across runs.	23 checks passed in 8 scenarios

9. Class Diagram (Textual)

9.1 Game and Levels

```
      +-------------------------------------------+ | Game | +-----------------------------------
```

9.2 Game State, Timer and Score Board

```
      +--------------------------------+ +--------------------------------+ | GameState | | Timer |
```

9.3 Data Structures and Algorithms

```
+-------------------+ +-------------------+ +-------------------+ | Stack
 | | Queue

  | | BST | +-------------------+ +-------------------+ +-------------------+ | - data_ | | - d

   | | Graph | | Algorithms | +-------------------+ +-------------------+ +-------------------+
```

Relationships: Game owns the five levels and uses ScoreBoard; every level shares the GameState; GameState contains the Timer; KeyVaultLevel uses Stack; WaitingGateLevel uses Queue and FinalEscapeLevel uses BST; MazeLevel and FinalEscapeLevel use Graph together with the functions in Algo; these functions are built on Queue (BFS), Stack (DFS) and MinHeap (Dijkstra).

10. GitHub Repository

The complete source code of the project is hosted in a GitHub repository:

Repository URL: https://github.com/your-username/codelock

Clone the repository, build with make and play:

```
git clone git@github.com:your-username/codelock.git cd codelock make ./codelock # runs the full
```

A C++17 compiler and make are required; the project was built and tested with g++ 12.2 on Linux.

11. Conclusion

CODELOCK achieves all of its stated objectives. It is a complete, playable game written only in C++, in which five puzzles are built on a Stack, a Queue, a Binary Search Tree, graph traversal and Dijkstra's algorithm. The data structures are implemented from scratch and each answer is checked by running them, so the game demonstrates how these structures solve practical problems instead of being isolated textbook exercises.

The design is modular: an abstract level class with one derived class per room, a shared game state, a separate user-interface module and header-only templates for the generic containers. The code compiles without warnings and is checked by unit tests and scripted play-throughs.

Possible future enhancements include randomly generated puzzles (the current puzzles are fixed), difficulty levels, a top-ten leaderboard, a live countdown running in its own thread (the clock is currently checked each time an answer is entered), and a graphical or web interface using a 2D graphics framework or an additional interface technology.

12. References

1. Stroustrup, B. (2013). *The C++ Programming Language*, 4th Edition. Addison-Wesley.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2022). *Introduction to Algorithms*, 4th Edition. MIT Press.
3. Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C++*, 4th Edition. Pearson.
4. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.
5. ISO/IEC 14882:2017. *Programming Languages — C++* (C++17 standard).
6. cppreference.com. C++ Reference. https://en.cppreference.com